Lab 6: Cache Implementation and Performance Analysis

CS24BT041

March 30, 2026

1 Introduction

In this lab, we implemented a memory hierarchy for the ToyRISC processor by adding L1 instruction (L1i) and L1 data (L1d) caches. The objective was to observe the impact of cache parameters (size and latency) on the overall processor performance, measured as Instructions Per Cycle (IPC).

2 Implementation Details

The cache system was integrated into the Discrete Event Simulator (DES) framework with the following characteristics:

- **Organization**: 2-way set-associative.
- **Line Size**: 4 bytes (1 word per line).
- **Replacement Policy**: Least Recently Used (LRU).
- **Write Policy**: Write-Through (for L1d).
- **Latency**: Variable based on cache size (1–4 cycles).

The pipeline stages (`InstructionFetch` and `MemoryAccess`) were modified to interface with their respective caches and stall until a `MemoryResponseEvent` is received.

3 Experimental Results

We performed two sets of experiments:

1. **Case 1**: Varying L1i cache size while keeping L1d fixed at 1024B.
2. **Case 2**: Varying L1d cache size while keeping L1i fixed at 1024B.

The benchmarks used include `descending`, `evenorodd`, `fibonacci`, `palindrome`, and `prime`.

3.1 Case 1: Varying L1i Cache Size

The following table summarizes the IPC for different L1i cache sizes. Note that increasing size also increases cache latency.

Table 1: Program IPC for Varying L1i (Fixed L1d = 1024B for sizes > 0; **0B column means no L1 caches**)

Benchmark	No Cache (0B)	16B (1cy)	128B (2cy)	512B (3cy)	1024B (4cy)
descending	0.0238	0.0233	0.0779	0.0741	0.0706
evenorodd	0.0236	0.0228	0.0224	0.0220	0.0216
fibonacci	0.0241	0.0233	0.0547	0.0525	0.0502
palindrome	0.0242	0.0236	0.0593	0.0572	0.0552
prime	0.0241	0.0292	0.0445	0.0429	0.0414

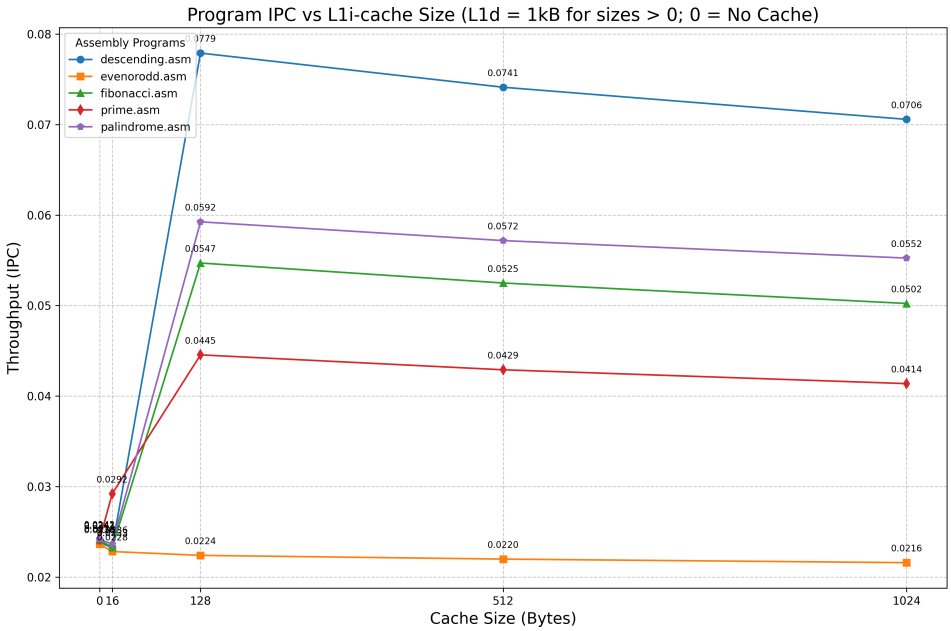

Figure 1: Program IPC vs L1i-cache Size

3.2 Case 2: Varying L1d Cache Size

The following table summarizes the IPC for different L1d cache sizes.

Table 2: Program IPC for Varying L1d (Fixed L1i = 1024B for sizes > 0; **0B column means no L1 caches**)

Benchmark	No Cache (0B)	16B (1cy)	128B (2cy)	512B (3cy)	1024B (4cy)
descending	0.0238	0.0608	0.0739	0.0722	0.0706
evenorodd	0.0236	0.0218	0.0217	0.0217	0.0216
fibonacci	0.0241	0.0512	0.0509	0.0506	0.0502
palindrome	0.0242	0.0554	0.0554	0.0553	0.0552
prime	0.0241	0.0415	0.0415	0.0414	0.0414

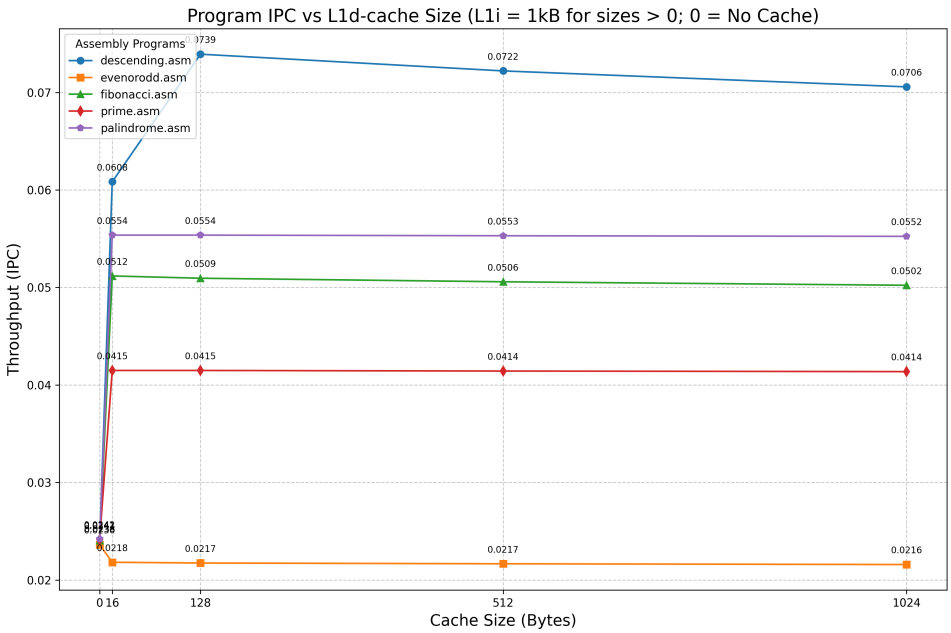

Figure 2: Program IPC vs L1d-cache Size

4 Observations and Conclusion

- **No-cache baseline**: Disabling both L1 caches (0B) yields the lowest IPC across all programs, confirming that the pipeline is sensitive to main-memory latency.
- **Case 1 (varying L1i)**: Some benchmarks (e.g., `descending`, `fibonacci`, `palindrome`) improve noticeably when moving from small `L1i` (16B) to larger capacities (128B), even though larger caches also have higher latency. Other benchmarks (e.g., `evenorodd`) show a mostly decreasing trend as latency increases.
- **Case 2 (varying L1d)**: Increasing `L1d` from 0B to 16B provides a large IPC jump for most programs, while further increases beyond 16B tend to plateau (since the remaining misses are dominated by instruction-fetch effects and/or the higher access latency).

- **Overall trade-off**: The results reflect the classic latency vs. hit-rate trade-off: cache capacity reduces misses, while larger caches increase access latency. The "best" size is benchmark-dependent.

In conclusion, the simulator shows that adding L1 caches improves throughput (IPC), but the optimal configuration depends on the benchmark. For data-heavy programs, increasing `L1d` capacity can significantly raise throughput; for instruction-stream behavior, `L1i` capacity and its latency are more influential.